

Lecture 7 Notes

Opening Administrative Remarks

The lecture begins with a brief administrative aside about calendars and an event or symposium schedule. The instructor mentions that sessions start around 9:00 and run until around 5:00, and that students can check the website or posters with QR codes for more information.

The instructor also notes that they are wearing a mask because of a health issue around the throat or voice area, but says their voice should still be clear.

Return to the Main Linear Algebra Problem

[Likely exam topic in transcript]

The lecture returns to the previous discussion: the central problem is the analysis of

$$\mathbf{Ax} = \mathbf{b}.$$

The instructor calls this the “mother of all problems” for the course. The main goals of the analysis are:

- Existence: does a solution exist?
- Uniqueness: if a solution exists, is it the only solution?
- Multiplicity: if it is not unique, how many solutions are there?

The course analyzes these questions using vector space methods.

Two Views of Matrix-Vector Multiplication

Before using vector spaces to analyze $\mathbf{Ax} = \mathbf{b}$, the instructor reviews two interpretations of multiplying a matrix by a vector.

For a matrix $\mathbf{A}$ multiplied on the right by a vector $\mathbf{x}$:

$$\mathbf{Ax}$$

there are two equivalent views:

1. Column-combination view:

$\mathbf{Ax}$ is a linear combination of the columns of $\mathbf{A}$, with the entries of $\mathbf{x}$ as coefficients.

2. Row-inner-product view:

Each entry of $\mathbf{Ax}$ is the inner product of $\mathbf{x}$ with one row of $\mathbf{A}$.

The same kind of interpretation can be made for the transpose matrix $\mathbf{A}^T$, with rows and columns interchanged.

These two perceptions connect matrix multiplication directly to the row space, column space, and the fundamental subspaces.

The Fundamental Spaces Picture

The lecture uses the “fundamental picture” of a matrix as a linear mapping.

For an $m \times n$ matrix $\mathbf{A}$,

$$\mathbf{A} : \mathbb{R}^n \rightarrow \mathbb{R}^m.$$

The input space is $\mathbb{R}^n$; the output or target space is $\mathbb{R}^m$. Four fundamental spaces are associated with this mapping:

- Column space / range space of $\mathbf{A}$, in the output space $\mathbb{R}^m$.
- Row space of $\mathbf{A}$, in the input space $\mathbb{R}^n$.

- Null space of A , in the input space $\mathbb{R}^n$.
- Left null space of A , in the output space $\mathbb{R}^m$.

The transcript repeatedly refers to the “green space”; from context this is the column space or range space, the set of all possible outputs of the map.

Column Space and Existence

The column space of A is:

$$\text{Col}(A) = \{Ax : x \text{ in } \mathbb{R}^n\}.$$

Equivalently, it is the span of the columns of A .

The column space is the set of all outputs that can be generated by the mapping $x \rightarrow Ax$. Therefore it answers the existence question for $Ax=b$:

- If $b \text{ in } \text{Col}(A)$, then at least one solution exists.
- If $b \text{ not in } \text{Col}(A)$, then no solution exists.

Important special case:

- If $\text{Col}(A) = \mathbb{R}^m$, then every possible right-hand side $b \text{ in } \mathbb{R}^m$ is reachable.
- In that case, a solution exists for every b .

This is marked in the transcript as important.

Null Space and Uniqueness

[Exam note in transcript]

The null space of A is:

$$N(A) = \{x \text{ in } \mathbb{R}^n : Ax = 0\}.$$

It is a subspace of the input space. The instructor emphasizes that the null space is orthogonal to the row space:

$$N(A) = \text{Row}(A)^\perp.$$

The null space controls uniqueness:

- If the null space is trivial, $N(A) = \{0\}$, then any solution is unique.
- If the null space contains any nonzero vector, then solutions cannot be unique.

Reason:

If x_p is one solution of $Ax=b$ and $z \text{ in } N(A)$, then

$$A(x_p + z) = Ax_p + Az = b + 0 = b.$$

So $x_p + z$ is also a solution. If there is any nonzero z in the null space, then there are infinitely many scalar multiples of z , and therefore infinitely many solutions whenever one solution exists.

The instructor also connects this to the orthogonal decomposition idea: a vector in the input space can be represented using components from the row space and its orthogonal complement, the null space.

Row Rank Equals Column Rank

[Exam note in transcript]

The instructor reviews the theorem:

$$\dim(\text{Row}(A)) = \dim(\text{Col}(A)).$$

The dimension of the column space is called the column rank:

$$R_c = \dim(\text{Col}(A)).$$

The dimension of the row space is called the row rank:

$$R_r = \dim(\text{Row}(A)).$$

The theorem says:

$$R_c = R_r.$$

This common dimension is called the rank of A :

$$r = \text{rank}(A).$$

Proof Idea Reviewed in Lecture

The instructor recalls the proof idea rather than fully redoing it.

One direction:

- Take a basis for the row space.
- Use the matrix mapping to show that the corresponding images are linearly independent in the column-space side.
- This gives an inequality of the form $R_c \geq R_r$.

The other direction:

- Apply the same argument to the transpose matrix A^T .
- Equivalently, start with a basis for the column space.
- This gives $R_r \geq R_c$.

Together:

$$R_c \geq R_r$$

and

$$R_r \geq R_c$$

so:

$$R_c = R_r.$$

This shared dimension is the matrix rank.

Rank Bounds

For an $m \times n$ matrix:

- $\text{Col}(A)$ is a subspace of $\mathbb{R}^m$, so $\text{rank}(A) \leq m$.
- $\text{Row}(A)$ is a subspace of $\mathbb{R}^n$, so $\text{rank}(A) \leq n$.

Therefore:

$$\text{rank}(A) \leq \min(m, n).$$

This upper bound leads to the distinction between full rank and rank-deficient matrices.

Full Rank and Rank Deficiency

The instructor defines the full rank case:

$$\text{rank}(A) = \min(m, n).$$

If this equality holds, A is full rank.

If not:

$$\text{rank}(A) < \min(m, n),$$

then the matrix is rank deficient. The transcript also uses the term “degenerate rank case” or “rank degenerate”; the instructor notes that “rank deficient” is the more common term.

Full rank has different implications depending on the shape of the matrix:

- For a tall matrix, full rank means the row space fills the input space.
- For a fat matrix, full rank means the column space fills the output space.
- For a square matrix, full rank means both happen at once.

Relationship Between Full Rank and Existence/Uniqueness

[Exam note in transcript]

The instructor uses the fundamental spaces picture to connect rank to existence and uniqueness.

Row Space Equals Input Space

If:

$$\text{Row}(A) = \mathbb{R}^n,$$

then:

$$N(A) = \{0\}.$$

This gives uniqueness whenever a solution exists.

This condition can be satisfied only for square or tall matrices, because the row space lies in $\mathbb{R}^n$ and needs enough independent rows to span it.

Column Space Equals Output Space

If:

$$\text{Col}(A) = \mathbb{R}^m,$$

then every $\mathbf{b}$ in $\mathbb{R}^m$ is reachable. This guarantees existence for every right-hand side $\mathbf{b}$.

This condition can be satisfied only for square or fat matrices, because the column space lies in $\mathbb{R}^m$ and needs enough independent columns to span it.

Non-Square Full Rank Matrices

For non-square matrices, full rank can satisfy only one of the two main conditions:

- A full rank tall matrix gives uniqueness, but not existence for every $\mathbf{b}$.
- A full rank fat matrix gives existence for every $\mathbf{b}$, but not uniqueness.

For square matrices, full rank satisfies both conditions.

Tall Matrices

Assume A is tall:

$$m > n.$$

There are more equations than unknowns.

The instructor suggests thinking of a tall matrix as having many rows. Because there are many row vectors, it is possible for the row space to span the whole input space $\mathbb{R}^n$, but this is not automatic.

Full Rank Tall Matrix

If A is tall and full rank, then:

$$\text{rank}(A) = n.$$

Consequences:

- $\text{Row}(A) = \mathbb{R}^n$.
- $N(A) = \{0\}$.
- If a solution exists, it is unique.

However, since the column space is a subspace of $\mathbb{R}^m$ with dimension at most n , and $n < m$, the column space cannot cover all of $\mathbb{R}^m$. Therefore:

- Existence is not guaranteed for arbitrary b .
- Some right-hand sides b will not lie in $\text{Col}(A)$.

So:

- If $b \in \text{Col}(A)$, there is a unique solution.
- If $b \notin \text{Col}(A)$, there is no solution.

Main point: for tall matrices, full rank is an indicator of uniqueness, not a guarantee of existence for every b .

Rank-Deficient Tall Matrix

If a tall matrix is rank deficient:

$$\text{rank}(A) < n.$$

Then:

- The row space does not fill $\mathbb{R}^n$.
- The null space is nontrivial.
- There are infinitely many vectors in the null space.

Therefore:

- If $b \in \text{Col}(A)$, then there are infinitely many solutions.
- If $b \notin \text{Col}(A)$, then there is no solution.

The instructor emphasizes that even though a tall matrix has many rows, linear dependence among the rows can prevent the row space from filling the input space.

Fat Matrices

Assume A is fat:

$$m < n.$$

There are more unknowns than equations.

The instructor says this is the opposite of the tall case: now there are many columns, so it may be possible for the column space to span the whole target space $\mathbb{R}^m$.

Full Rank Fat Matrix

If A is fat and full rank, then:

$$\text{rank}(A) = m.$$

Consequences:

- $\text{Col}(A) = \mathbb{R}^m$.
- The left null space is trivial.
- A solution exists for every b in $\mathbb{R}^m$.

But uniqueness is impossible for a fat matrix, because there are more unknowns than equations. The null space is nontrivial no matter what the rank is.

Therefore:

- For every b , there is at least one solution.
- For every b , there are actually infinitely many solutions.

Main point: for full rank fat matrices, existence for every b is guaranteed, but uniqueness is impossible.

Rank-Deficient Fat Matrix

If a fat matrix is rank deficient:

$$\text{rank}(A) < m.$$

Then:

- The column space does not span all of $\mathbb{R}^m$.
- The left null space is nontrivial.
- There are right-hand sides b outside the column space.

Therefore:

- If $b \notin \text{Col}(A)$, there is no solution.
- If $b \in \text{Col}(A)$, there are infinitely many solutions.

The instructor again notes terminology: “rank deficient” is the standard phrase, while “degenerate” means the matrix does not achieve the full possible rank $\min(m, n)$.

Square Matrices

Assume A is square:

$$m = n.$$

This is the only matrix shape where it is possible to satisfy both existence for every b and uniqueness at the same time.

Full Rank Square Matrix

If A is square and full rank:

$$\text{rank}(A) = m = n.$$

Consequences:

- $\text{Col}(A) = \mathbb{R}^m$.
- $\text{Row}(A) = \mathbb{R}^n$.
- The null space is trivial.
- The left null space is trivial.
- For every b , a solution exists.

- Since the null space is trivial, that solution is unique.

Such a matrix is invertible.

The unique solution is:

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}.$$

The instructor also says that in this case the matrix has a unique left inverse and a unique right inverse, and they are the same object, the inverse $\mathbf{A}^{-1}$.

Rank-Deficient Square Matrix

If a square matrix is rank deficient:

$$\text{rank}(\mathbf{A}) < n,$$

then:

- The row space does not fill the input space.
- The column space does not fill the output space.
- The null space is nontrivial.
- The left null space is nontrivial.

Therefore:

- For $\mathbf{b}$ in $\text{Col}(\mathbf{A})$, there are infinitely many solutions.
- For $\mathbf{b}$ not in $\text{Col}(\mathbf{A})$, there is no solution.

Example: Identical Rows or Columns

The instructor describes a rank-deficient square example where rows or columns are identical. In a 2×2 example, both the row space and column space may be one-dimensional lines inside a two-dimensional ambient space.

If $\mathbf{b}$ is not on the column-space line, there is no solution.

If $\mathbf{b}$ lies on the column-space line, solutions exist, but because the null space is nontrivial, there are infinitely many solutions.

Left and Right Inverses for Non-Square Matrices

The lecture briefly discusses one-sided inverses for non-square matrices.

For a full rank tall matrix $\mathbf{A}$ with shape $m \times n$, $m > n$, the natural one-sided inverse is a left inverse:

$$\mathbf{L} \mathbf{A} = \mathbf{I}_n.$$

For a full rank fat matrix $\mathbf{A}$ with shape $m \times n$, $m < n$, the natural one-sided inverse is a right inverse:

$$\mathbf{A} \mathbf{R} = \mathbf{I}_m.$$

The transcript is noisy in this part, but the conceptual point is that non-square full rank matrices can have one-sided inverses, while a full rank square matrix has a unique two-sided inverse.

For full rank square matrices:

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{I},$$

and

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}.$$

The same unique inverse is both a left and right inverse.

For non-square matrices, the side matters because the identity matrix has a different size on each side. A full rank tall matrix can satisfy $\mathbf{LA} = \mathbf{I}_n$, but it cannot generally satisfy $\mathbf{AR} = \mathbf{I}_m$; the latter would require spanning the larger output space. A full rank fat matrix can satisfy $\mathbf{AR} = \mathbf{I}_m$, but it cannot generally satisfy $\mathbf{LA} = \mathbf{I}_n$; the latter would require uniqueness in the larger input space.

The instructor also indicates that these one-sided inverses are not unique in the non-square full-rank cases:

- A full rank tall matrix generally has infinitely many left inverses.
- A full rank fat matrix generally has infinitely many right inverses.
- A full rank square matrix is the special case where the left and right inverse coincide and are unique.

When Uniqueness Is Useful and When Non-Uniqueness Is Useful

The instructor emphasizes that uniqueness is not always “good” and non-uniqueness is not always “bad.” It depends on the problem.

Estimation Example: Multi-User Signal Recovery

In an estimation problem, uniqueness is usually desired.

The instructor gives a communications example:

- Multiple users transmit signals.
- They transmit at the same time.
- They use the same frequency band.
- They interfere with one another.
- Such users are called co-channel users.
- A base station with multiple antennas receives mixtures of their signals.

Each antenna receives a linear combination of user signals. Conceptually:

$$y_i = h_{i1}x_1 + h_{i2}x_2 + \dots + h_{in}x_n.$$

Collecting all antenna observations gives a system:

$$\mathbf{y} = \mathbf{A}\mathbf{x}.$$

Here:

- $\mathbf{x}$ contains the individual transmitted user symbols or signals.
- $\mathbf{y}$ contains antenna observations.
- $\mathbf{A}$ contains channel coefficients, delays, gains, or related mixing parameters.

Goal:

Recover the actual transmitted user signals $\mathbf{x}$ without ambiguity.

In this estimation setting, infinitely many possible solutions are not useful. The receiver wants the actual transmitted symbols, not a set of equally valid candidates.

Without any prior assumptions on $\mathbf{x}$, the previous linear algebra analysis says:

- The number of equations must be at least the number of unknowns.
- In this communication example, the number of antenna observations should be at least the number of users.

In rough terms:

$$\text{number of antennas / observations} \geq \text{number of users}.$$

Prior Information Can Reduce Required Measurements

The instructor adds an important qualification:

The current analysis assumes no prior information about the source vector $\mathbf{x}$. It allows $\mathbf{x}$ to be any point in the full n -dimensional input space.

If additional structure is known, then fewer equations may be enough.

Examples of useful prior information:

- $\mathbf{x}$ lies in some restricted subset of $\mathbb{R}^n$.
- $\mathbf{x}$ is sparse: most entries are zero, but the zero locations are not known.
- The signals have other known structure, possibly such as coding or correlation properties.

For sparse vectors, the instructor says later techniques can recover the vector with high probability under assumptions on $\mathbf{A}$, even with fewer equations than unknowns.

The instructor says this will be discussed later and connects it to future homework or later course material. The current rank-based counting discussion assumes no prior information; the sparse or structured-signal case is a qualification, not the model being analyzed in this part of the lecture.

Control Example: Non-Uniqueness as Flexibility

The instructor contrasts estimation with control.

In a simple memoryless linear control system:

$$\mathbf{y} = \mathbf{A}\mathbf{x}.$$

Here:

- $\mathbf{x}$ is the input or control signal.
- $\mathbf{y}$ is the output.
- The goal is to choose $\mathbf{x}$ so that the system generates a desired target output $\mathbf{y}$.

If $\mathbf{A}$ is fat and full rank, then:

- Every target output $\mathbf{y}$ can be generated.
- For each target output, there are infinitely many inputs $\mathbf{x}$ that produce it.

In this setting, non-uniqueness is useful. It gives design freedom.

For example, among infinitely many inputs that generate the same target output, one can choose the input with smaller energy.

So:

- Estimation problems usually want uniqueness.
- Control problems can benefit from non-uniqueness.

The instructor frames control as a dual problem to estimation in this sense.

New Topic: Solving Linear Systems Through Matrix Factorization

[Exam note in transcript]

After finishing the analysis of existence and uniqueness, the lecture starts a new journey.

The stated goal is not mainly to study numerical algorithms in detail. Instead, solving $\mathbf{A}\mathbf{x}=\mathbf{b}$ is used as a motivation or excuse to introduce important matrix factorizations.

The central strategy:

Convert a hard system of linear equations into a sequence of simple problems.

The instructor emphasizes that this picture will lead to many important results in linear algebra.

Hard Problems and Simple Problems

For an arbitrary matrix A , solving:

$$Ax = b$$

can be hard because the equations are coupled. Each equation may involve many or all unknowns.

The instructor warns that being square, or even having some special-looking structure such as symmetry, is not by itself the notion of “simple” being used here. The difficulty is coupling: if each equation mixes many unknowns, then equal numbers of equations and unknowns do not automatically make the system easy.

The idea is to rewrite or transform the problem so that it becomes a sequence of problems involving special simple matrices.

This is exactly what matrix factorization does:

$$A = A_1 A_2 \dots A_k,$$

where each factor A_i is a simple matrix.

Then:

$$Ax = b$$

becomes:

$$A_1 A_2 \dots A_k x = b,$$

which can be solved step by step.

The instructor says that famous factorizations are essentially different ways of writing a matrix as a product of simple matrices.

Simple Matrix Type 1: Diagonal Matrices

The simplest case is a diagonal matrix.

For a diagonal matrix:

```
D =
\begin{bmatrix}
d_1 & 0 & \cdots & 0 \\
0 & d_2 & \cdots & 0 \\
\vdots & \vdots & \ddots & \vdots \\
0 & 0 & \cdots & d_n
\end{bmatrix},
```

the system:

$$Dx = b$$

is uncoupled:

$$d_i x_i = b_i.$$

Each unknown can be solved independently:

$$x_i = b_i / d_i.$$

For a square diagonal matrix to be full rank, every diagonal entry must be nonzero:

$$d_i \neq 0 \quad \text{for all } i.$$

The instructor notes that this is simple because there is no coupling among the variables.

Student Remark: Decoupled Does Not Have to Mean Diagonal

A student points out that the nonzero entries need not literally be on the main diagonal for the system to be easy. If each equation involves only one variable and each variable appears separately, the system is still decoupled, even if the nonzero entries are permuted away from the diagonal.

The instructor agrees. Such a matrix can be represented using a diagonal matrix times a permutation matrix.

This leads naturally to permutation matrices, discussed later as a special case of orthogonal matrices.

Simple Matrix Type 2: Triangular Matrices

The second simple case is a triangular matrix.

Gaussian elimination aims to transform a general system into triangular form, often an upper triangular or row echelon form.

Upper Triangular Case

For an upper triangular matrix U , all entries below the diagonal are zero.

In a system:

$$Ux = b,$$

the last row involves only the last unknown x_n . So:

1. Solve the last equation for x_n .
2. Move one row up.
3. That row involves x_{n-1} and x_n , but x_n is already known.
4. Substitute the known value of x_n and solve for x_{n-1} .
5. Continue upward.

This procedure is called back substitution.

The instructor says Gaussian elimination tries to reach triangular form and then uses back substitution to solve the system.

Solving a triangular system is equivalent to applying the inverse triangular matrix, but the instructor stresses it as an iterative substitution procedure rather than explicitly forming the inverse.

Lower Triangular Case

For a lower triangular matrix L , all entries above the diagonal are zero.

The solution order is the reverse of the upper triangular case:

1. Start from the first row.
2. Solve for x_1 .
3. Move to the second row and solve for x_2 , using the known value of x_1 .
4. Continue downward.

This is the dual of the upper triangular case. The standard name is forward substitution, although the transcript mostly describes it rather than naming it.

Simple Matrix Type 3: Orthogonal Matrices

[Exam note in transcript]

The third simple case is an orthogonal matrix. The transcript sounds like “alternate” in places, but the mathematical condition described is the orthogonal condition.

For a real orthogonal matrix Q :

$$Q^{-1} = Q^T.$$

Equivalently:

$$Q^T Q = I.$$

This makes solving:

$$Qx = b$$

easy:

$$x = Q^T b.$$

The inverse is cheap to compute because it is just the transpose.

The instructor notes that for complex matrices the definition must be modified; the corresponding operation would use the conjugate transpose rather than the ordinary transpose.

Permutation Matrices

Permutation matrices are examples of orthogonal matrices.

Example:

```
P =
\begin{bmatrix}
0 & 1 \\
1 & 0
\end{bmatrix}.
```

This matrix swaps two components. It is symmetric in this example:

$$P^T = P.$$

It is also orthogonal:

$$P^T P = I.$$

Permutation matrices can reorder rows or columns. The instructor connects them to the earlier student remark: a system that is decoupled but not diagonal can be viewed as a diagonal system combined with a permutation.

Famous Factorizations as Products of Simple Matrices

The instructor says that famous matrix factorizations are built from combinations of these simple matrix types:

- Diagonal matrices.
- Triangular matrices.
- Orthogonal matrices.
- Permutation matrices.

The general theme is:

hard matrix = product of simple matrices.

Then:

hard problem = sequence of simple problems.

Singular Value Decomposition

[Exam note in transcript]

The lecture mentions singular value decomposition as one example of this philosophy:

$$A = U \Sigma V^T.$$

Here:

- U is orthogonal.
- V is orthogonal.
- Σ is diagonal, or diagonal-like in the rectangular case.

So SVD writes a general matrix as a product of simple matrices.

The instructor says this is part of the “story of all factorizations.”

Symmetric Matrix Decomposition

For symmetric matrices, the lecture mentions a decomposition of the form:

$$A = Q \Lambda Q^T.$$

Here:

- Q is orthogonal.
- Λ is diagonal.

Again, the matrix is converted into simple pieces.

This foreshadows diagonalization and eigenvalue-related topics.

QR Factorization

The lecture then focuses on QR factorization as an example of turning one hard problem into two simple ones.

In QR factorization:

$$A = QR.$$

where:

- Q is orthogonal.
- R is upper triangular.

The instructor states that such a factorization can be obtained for matrices under the conditions to be discussed later.

Solving $Ax=b$ Using QR

Start with:

$$Ax = b.$$

If:

$$A = QR,$$

then:

$$QRx = b.$$

Define:

$$y = Rx.$$

Then the original system becomes two systems:

$$Qy = b$$

and

$$Rx = y.$$

Because Q is orthogonal:

$$y = Q^T b.$$

Then solve:

$$Rx = y$$

by back substitution, because R is upper triangular.

So the hard problem $Ax=b$ becomes:

1. Multiply by Q^T to solve the orthogonal part.
2. Use back substitution to solve the triangular part.

This is the core factorization perspective: factorization converts one hard system into a sequence of simple systems.

Software Remark: MATLAB Backslash Operator

The instructor makes an extended remark about software.

In MATLAB, writing:

$$x = A \backslash b$$

solves a system of linear equations. The instructor says it looks like division, but that appearance is misleading. It is actually a powerful operator that chooses appropriate linear algebra methods, often based on factorizations.

The instructor notes that the backslash operator is especially powerful because it also handles non-square systems and can return special solutions. The transcript says something like “minimum norm”; in underdetermined systems, this means a solution with minimum norm, while in overdetermined systems such tools often relate to least-squares solutions.

The instructor contrasts MATLAB with Python, where one typically has to call a linear algebra function more explicitly, such as from NumPy or SciPy. The exact Python function is not clearly stated in the transcript.

There is also a joking remark that one could ask Codex to solve the system in MATLAB or Python, but the instructor does not want to spend months debugging code.

LU Factorization and Gaussian Elimination

The next example is LU factorization:

$$A = LU.$$

Here:

- L is lower triangular.
- U is upper triangular.

The instructor notes a qualification:

Not every matrix can be written directly as LU without additional operations. Sometimes a permutation matrix is needed, for row exchanges or pivoting. This leads to forms such as a permutation times A being factored into L and U .

The core idea remains the same: A is represented as a product of simple matrices.

The instructor identifies Gaussian elimination as the process of finding lower and upper triangular factors.

Solving $Ax=b$ Using LU

If:

$$A = LU,$$

then:

$$LUx = b.$$

Define:

$$y = Ux.$$

Then solve two triangular systems:

$$Ly = b$$

and

$$Ux = y.$$

Since L is lower triangular, solve $Ly=b$ from top to bottom.

Since U is upper triangular, solve $Ux=y$ from bottom to top using back substitution.

Again, a hard system is transformed into a sequence of simple systems.

At the comparison level, QR and Gaussian elimination both aim to reach an upper triangular problem, but they use different simple transformations. QR uses orthogonal matrices together with an upper triangular factor; Gaussian elimination uses lower triangular row-operation matrices to produce an upper triangular matrix.

Gaussian Elimination as Row Operations

[Exam note in transcript]

The instructor connects LU factorization to row operations from homework zero.

The goal of Gaussian elimination is to eliminate entries below the diagonal to obtain an upper triangular matrix.

For example, suppose the first column has entries below the first pivot. To eliminate them, one might use operations such as:

$$R_2 \rightarrow R_2 - 2R_1$$

and

$$R_3 \rightarrow R_3 - 3R_1.$$

After these operations, the entries below the first pivot are zero. This is described as the “first victory” toward an upper triangular matrix.

Then one proceeds to the next column. To eliminate the entry below the second pivot, use row 2 rather than row 1, because using row 1 again may destroy zeros already created.

The instructor emphasizes the algorithmic nature of the row replacement notation.

Row Operations as Left Multiplication

The instructor makes an important conceptual point:

Row operations are matrix multiplications.

Specifically, row operations correspond to left multiplication by another matrix.

Reason:

- A row operation forms new rows as linear combinations of old rows.
- Left multiplication takes linear combinations of rows.
- Right multiplication would take linear combinations of columns.

So Gaussian elimination can be expressed as:

$E A,$

where E is an elimination matrix.

Example of an Elimination Matrix

For the operations:

$R_2 \rightarrow R_2 - 2R_1$

and

$R_3 \rightarrow R_3 - 3R_1,$

the corresponding left-multiplying matrix has rows:

```
\begin{bmatrix}
1 & 0 & 0 \\
-2 & 1 & 0 \\
-3 & 0 & 1
\end{bmatrix}.
```

The first row $[1,0,0]$ means row 1 is unchanged.

The second row $[-2,1,0]$ means:

$\text{new } R_2 = -2 R_1 + R_2.$

The third row $[-3,0,1]$ means:

$\text{new } R_3 = -3 R_1 + R_3.$

This matrix is lower triangular.

Causality Rule in Gaussian Elimination

The instructor describes a “causality” rule in Gaussian elimination.

Standard elimination uses earlier rows, with lower row index, to modify later rows, with higher row index.

For example:

- Use row 1 to eliminate entries in rows 2 and 3.
- Then use row 2 to eliminate entries in lower rows.
- Do not use a lower row to change an earlier row during ordinary forward elimination.

This rule is what makes the elimination matrices lower triangular.

In a lower triangular left-multiplying matrix:

- Row 1 can use only row 1.

- Row 2 can use rows 1 and 2.
- Row 3 can use rows 1, 2, and 3.

There are no entries above the diagonal because earlier output rows do not use later input rows.

This is a key relationship:

Gaussian elimination row operations $\leftrightarrow$ lower triangular elimination matrices.

From Elimination Matrices to LU

Through Gaussian elimination, one applies a sequence of lower triangular elimination matrices:

$$E_k \cdots E_2 E_1 A = U.$$

Here U is upper triangular.

This equation is not yet written as a factorization of A . To express A itself, move the elimination matrices to the other side by taking inverses:

$$A = E_1^{-1} E_2^{-1} \cdots E_k^{-1} U.$$

The product:

$$L = E_1^{-1} E_2^{-1} \cdots E_k^{-1}$$

is lower triangular, so:

$$A = LU.$$

Why L Is Lower Triangular

The instructor gives two structural facts:

1. The inverse of a lower triangular matrix is lower triangular.
2. The product of lower triangular matrices is lower triangular.

Therefore, when we multiply inverses of elimination matrices together, the result is still lower triangular.

The elimination matrices used in the lecture are also unit lower triangular: their diagonal entries are 1. This reflects that a row is kept with coefficient 1 while multiples of earlier rows are added or subtracted below the diagonal. Their inverses remain unit lower triangular.

The transcript also notes that for the special elimination matrices used in Gaussian elimination, the signs of the below-diagonal multipliers flip when taking inverses. For example, elimination coefficients that appear as negative values in E become positive entries in E^{-1} and hence in L .

This is why Gaussian elimination using lower triangular elimination matrices leads to an LU factorization.

Row Exchanges and Permutation Matrices

Sometimes Gaussian elimination requires exchanging rows, either at the beginning or during the process. This is often needed to avoid a zero or bad pivot.

Row exchanges are represented by permutation matrices.

A row-swap permutation matrix is orthogonal:

$$P^T P = I.$$

Example: a permutation matrix that swaps rows 1 and 2 makes:

- the first row of the output equal to the second row of the input,
- the second row of the output equal to the first row of the input.

Because of row exchanges, a more general factorization may involve a permutation matrix together with L and U, rather than a plain $A=LU$.

The transcript mentions left and right multiplication by permutation matrices; for row exchanges specifically, the relevant operation is left multiplication.

Lower Triangular Matrices and Causal Systems

The instructor highlights a system-theoretic interpretation of lower triangular matrices.

If the indices represent time, a lower triangular matrix corresponds to a causal system.

For example:

$$y = Lx.$$

For a three-sample causal system, the lower triangular form is:

```
\begin{bmatrix}
y_1 \\
y_2 \\
y_3
\end{bmatrix}
=
\begin{bmatrix}
h_{11} & 0 & 0 \\
h_{21} & h_{22} & 0 \\
h_{31} & h_{32} & h_{33}
\end{bmatrix}
\begin{bmatrix}
x_1 \\
x_2 \\
x_3
\end{bmatrix}.
```

With a lower triangular L:

y_1

depends only on:

x_1 .

Similarly:

y_2

can depend on:

x_1, x_2 .

And:

y_3

can depend on:

x_1, x_2, x_3 .

But no output at time i depends on a future input x_j with $j > i$.

This is exactly the causality condition:

present output does not depend on future input.

The instructor says this is one reason lower triangular matrices have a special place in system theory.

Homework Remarks

[Exam note in transcript]

The instructor refers to “homework zero” when discussing Gaussian elimination row operations.

The instructor also says “do homework” twice near the discussion of lower triangular matrices and causal systems, implying that the related homework reinforces these ideas and should work out if done.

Factorization Summary From This Lecture

The lecture’s main conceptual chain is:

1. Start with a hard linear system:

$$\mathbf{Ax} = \mathbf{b}.$$

2. Recognize that some matrices make systems easy:

- Diagonal matrices give uncoupled equations.
- Triangular matrices can be solved by substitution.
- Orthogonal matrices are inverted by transposition.
- Permutation matrices just reorder rows or columns.

3. Factor a hard matrix into simple factors:

$$\mathbf{A} = \mathbf{A}_1 \mathbf{A}_2 \cdots \mathbf{A}_k.$$

4. Replace the hard system with a sequence of simple systems.

5. This viewpoint leads to QR, LU, SVD, and diagonalization-type factorizations.

Next Lecture Preview

The instructor says the next lecture will continue this story.

The next question will be whether one can use smart transformations to convert a matrix into diagonal form. Diagonal form is especially desirable because diagonal systems are the easiest to solve.

This foreshadows diagonalization and related matrix decompositions.

The instructor also mentions that there will be no class on Tuesday and asks students to attend or register for the symposium. The instructor plans to send the symposium website.

Source and Coverage Note

These notes were created only from `corrected/lecture7_corrected.md`. They preserve the lecture’s chronological flow and include the transcript’s marked exam notes, definitions, proof ideas, examples, instructor remarks, warnings about terminology, software comments, homework reminders, and relationships among the concepts. Some noisy transcript phrases were interpreted in context, especially “green space” as column/range space and “alternate” as orthogonal matrix; where the wording was uncertain, the surrounding explanation was retained rather than compressed away.